

QUANT FINANCE TERMINAL ·
v2.046

SYS://CURRICULUM/D035.MD · PHASE 2 ●
LIVE

LECTURE · 量化金融 365

DAY 035 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

时间索引与重采样

// DatetimeIndex

KEY CONCEPTS · 三大要点

- ▶ DatetimeIndex
- ▶ resample
- ▶ 时区

01 · WHY THIS LESSON · 这节课为什么重要

本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

今天讲 Pandas 处理金融数据的看家本领 — 时间索引与重采样。为什么单拎出来讲?因为时间是金融数据的灵魂。股票数据跟普通表格最大的不同,就是它的每一行都死死绑定着一个时间点 — 几月几号、几点几分。把时间这个维度玩明白了,你才算真正会处理行情数据。这一节做五件事 — 第一,讲 DatetimeIndex,Pandas 专门为时间设计的索引,它能让你写一句「给我 12 月的数据」就自动筛出整个 12 月;第二,讲 `pd.to_datetime`,怎么把行情软件导出的五花八门的日期字符串,统一解析成标准时间戳;第三,讲今天的主角 `resample` 重采样 — 它能把 5 分钟的高频数据,聚合成日线、周线、月线,这是量化里天天用的操作;第四,讲 `resample` 里最容易踩的坑 — 不同的列要用不同的聚合方式,开盘价取第一个、最高价取最大、成交量要求和,绝不能一刀切全取平均;第五,讲时区处理,A 股是北京时间、美股是纽约时间,做跨市场分析时怎么把它们对齐到同一时刻。我们用真实的茅台 5 分钟线,把分钟数据亲手聚合成日线,所有操作都在真数据上跑一遍。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

- 01 掌握 DatetimeIndex 时间索引 — 它支持「部分字符串索引」,写一句日期或月份就能筛出对应数据,还能直接从索引取出小时、分钟做日内分析
- 02 学会 pd.to_datetime — 把各种格式的日期字符串统一解析成标准时间戳,理解格式参数的作用和月日顺序这类常见解析坑
- 03 吃透 resample 重采样 — 它是「时间版的 groupby」,能把分钟聚合成日线、日线聚合成周线月线,是量化处理行情最高频的操作
- 04 记牢 OHLCV 聚合规则 — 开盘取第一个、最高取最大、最低取最小、收盘取最后一个、成交量要求和,绝不能所有列一刀切取平均,这是新手最大的坑
- 05 理解时区处理 — tz_localize 是给时间贴时区标签、tz_convert 是换算到另一个时区看同一时刻,做 A 股加美股的跨市场对齐前必须先统一时区

02 · CONTEXT · 历史与背景

小陈把成交量也取了平均,量价信号全错还浑然不觉

// 不读历史的策略走不远

讲一个量化新手小陈的真实翻车。小陈想做一个「放量突破」策略——当某天成交量明显放大、同时价格突破近期高点时买入。他手里有茅台的5分钟数据,需要先把它聚合成日线,才能算「每天的成交量」和「每天的收盘价」。

他知道 Pandas 有个 `resample` 函数能做聚合,于是图省事,写了一句 `df.resample('D').mean()`——把所有列都取了平均。代码不报错,跑得很顺,他就以为大功告成了。

问题大了。「取平均」对成交量是致命错误。一天有几十根5分钟K线,每根都有自己的成交量,一天真实的总成交量应该是把这几十根的量全部加起来(求和),而他取了平均,等于把总量除以了几十,数字一下子小了几十倍。更糟的是,收盘价取平均也错了——一天的收盘价应该是最后一根K线的价格,而不是全天价格的平均值。结果他算出来的「每天成交量」全是缩水几十倍的假数据,「每天收盘价」也是糊成一团的平均值。

基于这堆错数据,他的「放量突破」策略自然永远触发不了正确信号,回测结果一塌糊涂。他查了3天代码,逻辑部分一个bug都没有,百思不得其解,最后才发现根子在最开始那一句 `resample` 用错了聚合方式。

这个故事的教训极其重要:做时间重采样时,不同的列必须用不同的聚合方式。这套规则在金融里有个标准名字,叫 OHLCV 聚合——开盘价 `Open` 取这段时间的第一个、最高价 `High` 取最大、最低价 `Low` 取最小、收盘价 `Close` 取最后一个、成交量 `Volume` 要求和。价格类绝不能取平均,成交量绝不能取最后一个。一刀切是新手最常见、也最隐蔽的错误,因为代码不报错,你不盯着结果根本发现不了。

今天我们就用真实的茅台5分钟数据,把正确的 OHLCV 聚合亲手跑一遍,再故意演示一遍「全取平均」的错误后果,让你对这个坑刻骨铭心。学完这一节,你就掌握了把任意高频数据转成任意周期K线的核心能力,这是写任何策略的第一步。

把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

CONCEPT_01

DatetimeIndex — 给时间装上超能力的专用索引

昨天我们的 DataFrame 行标签虽然是日期,但只是把它当普通文字标签用。今天要把日期升级成 DatetimeIndex —— Pandas 专门为时间打造的索引类型,一旦升级,会解锁一堆超能力。

第一个超能力叫「部分字符串索引」。普通索引你要取某天数据,得精确报出完整标签。而 DatetimeIndex 聪明得多:你写 `df.loc['2024-12']`,它自动把整个 12 月的所有行都筛出来;你写 `df.loc['2024-12-02']`,它把那一天的所有分钟 K 线都筛出来。给个年份选一整年、给个月份选一整月、给一天选一整天,完全符合人的直觉,一句话搞定,不用写复杂的筛选条件。

第二个超能力是「时间属性直取」。DatetimeIndex 的每个时间点都知道自己是几年几月几号、星期几、几点几分。你可以直接从索引里取出小时(`index.hour`)、星期几(`index.dayofweek`),用来做日内分析或者剔除周末。比如想看「每天开盘第一个小时的成交量」,直接按小时分组就行。

第三个超能力是「时间感知的运算」,也就是后面要讲的 `resample` 和时区转换,全都建立在 DatetimeIndex 之上。普通文字索引做不了这些。

怎么得到 DatetimeIndex?把你的日期/时间列用 `pd.to_datetime` 解析一下,再 `set_index` 设成索引就行。这是处理任何行情数据的标准第一步。记住:行情数据进门,先把时间列转成 DatetimeIndex,后面一切才方便。

CONCEPT_02

pd.to_datetime — 把五花八门的时间字符串理顺

真实世界的时间数据格式乱得很。有的写成 2024-12-02,有的写成 2024/12/02,有的写成 12/02/2024,还有的像 baostock 这样写成一长串数字 20241202093500000(年月日时分秒毫秒糊在一起)。pd.to_datetime 就是把这些五花八门的字符串,统一解析成 Pandas 能理解的标准时间戳。

大多数常见格式,pd.to_datetime 能自动识别,直接喂给它就行。但碰到不标准的格式(比如那串糊在一起的数字),就需要用 `format` 参数明确告诉它怎么拆 —— 哪几位是年、哪几位是月、哪几位是时分秒。指定了 `format` 不仅解析更准,速度也快很多,处理大数据时差别明显。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT_03

resample 重采样 — 时间版的 groupby

resample 是今天的绝对主角,也是量化处理行情数据用得最多的操作之一。一句话理解它:resample 就是「时间版的 groupby 分组」。

普通的 groupby 是按某个类别分组(比如按行业把股票分成几堆)。而 resample 是按时间段分组——把数据按你指定的周期(每天、每周、每月)切成一段一段,然后对每一段做聚合。最典型的用途就是「降频」:把5分钟的高频数据,聚合成日线;把日线聚合成周线、月线。这正是开头小陈要做的事。

用法很直观:df.resample('D') 表示「按天分组」,'W' 是按周,'ME' 是按月末。分组之后,跟 groupby 一样要告诉它每组怎么聚合——求和用 sum、取最后一个用 last、取最大用 max。比如把分钟收盘价聚合成日收盘价,就是 df['close'].resample('D').last(),意思是「每天取最后一根 K 线的收盘价」。

几个实用细节。第一,resample 按自然周期切,周末、节假日这些没数据的日子会产生空行 (NaN),记得用 dropna 清掉。第二,周期标签有讲究,'ME' 是月末对齐、'MS' 是月初对齐,用错了日期会差半个月,要按需求选。第三,resample 还能「升频」(把日线插值成分钟),但量化里很少用,主要还是降频。

掌握了 resample,你就能把任意高频数据转成任意周期的 K 线,这是做任何周期策略的基本功。但它有个大前提——不同的列要用不同的聚合方式,这就是下一个概念。

CONCEPT_04

OHLCV 聚合规则 — 不同列必须用不同的聚合方式

这是今天最重要、也最容易踩的实战要点,开头小陈翻车的根源就在这里。当你把高频数据 resample 成低频时,行情数据的五个标准字段——开盘价 Open、最高价 High、最低价 Low、收盘价 Close、成交量 Volume(合称 OHLCV)——每一个都有它专属的、不能搞错的聚合方式。

逐个来说。开盘价 Open 取这段时间的「第一个」值——一天的开盘价就是当天第一根 K 线的开盘价,用 first。最高价 High 取「最大」值——一天的最高价是当天所有 K 线里最高的那个,用 max。最低价 Low 取「最小」值,用 min。收盘价 Close 取「最后一个」值——一天的收盘价是当天最后一根 K 线的收盘价,用 last。成交量 Volume 取「求和」——一天的总成交量是当天所有 K 线成交量的总和,用 sum。

03 · CORE CONCEPTS · 续 (3/3)

CONCEPT_05

时区与工作日偏移 — 跨市场对齐的隐形地基

最后讲时区。平时只做 A 股可能感觉不到时区的存在,但一旦你要做跨市场分析 —— 比如研究 A 股和美股的联动 —— 时区就成了一个绕不开的隐形地基,处理不好会得出完全错误的结论。

核心是两个容易混淆的操作。第一个是 `tz_localize`,「贴时区标签」。刚解析出来的时间是「裸」的(*naive*),不带任何时区信息,只是一串钟点数字。`tz_localize` 的作用是告诉 Pandas「这串钟点是哪个时区的」,比如 A 股数据全是北京时间,就 `tz_localize('Asia/Shanghai')`。注意它只是贴个标签,钟点数字本身不变。第二个是 `tz_convert`,「换算时区」。它把一个已经带时区的时间,换算成另一个时区下「同一时刻」的钟点。比如北京时间上午9:30,换算成纽约时间就成了前一天的晚上8:30(钟点变了,但指的是宇宙中同一个瞬间)。

这俩为什么重要?做 A 股加美股的跨市场对齐时,如果不统一时区,你会把北京时间的9:30和纽约时间的9:30当成同一时刻去对齐,实际上它们差了十几个小时,数据全错位 —— 这跟之前讲过的跨市场对齐陷阱一脉相承。正确做法是:两边数据各自 `localize` 到自己的时区,再统一 `convert` 到同一个时区(比如都转成 UTC),然后才能放心对齐。

顺带讲一个实用的小工具:工作日偏移 `BusinessDay`。金融计算经常要「往后推一个交易日」,但不能简单加一天(可能撞上周末)。`pd.offsets.BDay` 能自动跳过周末,帮你正确地在工作日之间移动。这在算「T 加一交割」「信号次日执行」这类逻辑时特别有用。一句话:`localize` 贴标签、`convert` 换时区,跨市场先统一时区;工作日移动用 `BDay` 自动跳周末。

04 · PYTHON LAB · 真实可跑代码

时间索引与重采样五件套 — to_datetime / DatetimeIndex / resample 分钟转日线 / OHLCV 聚合 / 时区

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_035_datetime_resample.py — 时间索引与重采样: DatetimeIndex / to_datetime /  
resample / OHLCV 聚合 / 时区  
# 用真实A股5分钟线,把分钟数据resample成日线/周线,讲透时间是金融数据的灵魂  
# 数据:baostock 5分钟线(免费、稳定、国内零翻墙;A股原生北京时间)  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import baostock as bs  
  
pd.set_option('display.width', 140)  
CODE = 'sh.600519' # 茅台,用一只票把分钟线讲透  
NAME = '茅台'  
START = '2024-11-01'  
END = '2024-12-31'  
  
# ===== 0. 数据拉取:5分钟线 OHLCV =====  
print('==== 0. 数据拉取(baostock 5分钟线)====')  
lg = bs.login()  
if lg.error_code != '0':  
    raise RuntimeError(f'baostock 登录失败:{lg.error_msg}')  
rs = bs.query_history_k_data_plus(  
    CODE, 'date,time,open,high,low,close,volume',  
    start_date=START, end_date=END, frequency='5', adjustflag='2')  
rows = []  
while (rs.error_code == '0') and rs.next():
```


04 · PYTHON LAB · 真实可跑代码 · 续 (2/5)

时间索引与重采样五件套 — to_datetime / DatetimeIndex / resample 分钟转日线 / OHLCV 聚合 / 时区

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
rows.append(rs.get_row_data())
bs.logout()
raw = pd.DataFrame(rows, columns=['date', 'time', 'open', 'high', 'low', 'close',
'volume'])
print(f'{NAME} 5 分钟原始数据 {len(raw)} 条')

# ===== 1. pd.to_datetime:把字符串解析成时间戳 =====
print('\n=== 1. pd.to_datetime 解析时间 ===')
# baostock 的 time 字段形如 '20241101093500000' (年月日时分秒毫秒)
print('原始time 字段样例:', raw['time'].iloc[0])
ts = pd.to_datetime(raw['time'], format='%Y%m%d%H%M%S%f')
df = raw.copy()
for c in ['open', 'high', 'low', 'close', 'volume']:
    df[c] = pd.to_numeric(df[c], errors='coerce')
df = df.set_index(ts).drop(columns=['date', 'time']).sort_index()
df.index.name = '时间'
print(f'解析后索引类型 = {type(df.index).__name__}')
print(f'时间范围 {df.index.min()} → {df.index.max()}')

# ===== 2. DatetimeIndex:部分字符串索引 + .时间属性 =====
print('\n=== 2. DatetimeIndex 的超能力 ===')
# 部分字符串索引:给一个日期就选出那天所有分钟
one_day = df.loc['2024-12-02']
print(f'df.loc["2024-12-02"] 选出一整天 {len(one_day)} 根 5 分钟K 线(部分字符串索引)')
# 给一个月份选出整月
one_month = df.loc['2024-12']
print(f'df.loc["2024-12"] 选出整个 12 月 {len(one_month)} 根K 线')
# 直接从索引拿小时/分钟, 做日内分析
```

```
print('索引自带时间属性 index.hour 样例:', sorted(set(df.index.hour)))
```

```
# ===== 3. resample:时间版的 groupby(分钟 → 日线)=====
```

04 · PYTHON LAB · 真实可跑代码 · 续 (3/5)

时间索引与重采样五件套 — to_datetime / DatetimeIndex / resample 分钟转日线 / OHLCV 聚合 / 时区

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print('\n==== 3. resample 分钟 → 日线 ====')
# 核心:不同列聚合方式不同!开=first 高=max 低=min 收=last 量=sum
daily = df.resample('D').agg({
    'open': 'first', 'high': 'max', 'low': 'min', 'close': 'last', 'volume': 'sum',
}).dropna(how='any')
print(f'{len(df)} 根 5 分钟线 → {len(daily)} 根日线')
print('日线前 3 行:')
print(daily.head(3).round(2))
# 校验:某天日线的 high 必须等于那天所有分钟 high 的最大值
chk_day = '2024-12-02'
minute_high = df.loc[chk_day, 'high'].max()
daily_high = daily.loc[chk_day, 'high']
print(f'校验 {chk_day}:分钟最高 {minute_high:.2f} == 日线 high {daily_high:.2f} ?',
      np.isclose(minute_high, daily_high))

# ===== 4. OHLCV 聚合规则:价格不能取平均、量不能取 last =====
print('\n==== 4. 聚合规则对错对比 ====')
# 错误示范:对所有列一律取平均
wrong = df.resample('D').mean().dropna(how='any')
# 对比某天:正确收盘(last) vs 错误收盘(mean)
right_close = daily.loc[chk_day, 'close']
wrong_close = wrong.loc[chk_day, 'close']
print(f'{chk_day} 收盘价 - 正确(取最后一根)= {right_close:.2f}')
print(f'{chk_day} 收盘价 - 错误(取全天平均)= {wrong_close:.2f} ← 差了 {abs(right_close-wrong_close):.2f} 元,信号全错')
# 成交量:必须 sum 不能 mean
vol_sum = df.loc[chk_day, 'volume'].sum()
vol_mean = df.loc[chk_day, 'volume'].mean()
print(f'{chk_day} 成交量 - 正确(求和)= {vol_sum:,.0f} 错误(平均)= {vol_mean:,.0f}(差 {vol_sum/vol_mean:.0f} 倍)')
```

```
# resample 到周线  
weekly = df['close'].resample('W').last().dropna()
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/5)

时间索引与重采样五件套 — to_datetime / DatetimeIndex / resample 分钟转日线 / OHLCV 聚合 / 时区

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print(f'再 resample 到周线:{len(weekly)} 根')

# ===== 5. 时区:tz_localize 贴标签 vs tz_convert 换时区 =====
print('\n==== 5. 时区处理 ====')
naive = df.index[:3]
print('原始时间不带时区(naive):', list(naive.astype(str)))
# A 股是北京时间,先「贴上」时区标签(localize,不改钟点只贴标签)
aware = df.index.tz_localize('Asia/Shanghai')
print('贴上北京时区后:', str(aware[0]))
# 换算成美东时间「同一时刻」(convert,钟点会变)
us = aware.tz_convert('America/New_York')
print('换算成纽约时间(同一时刻):', str(us[0]))
print('教训:localize 只贴标签不改钟点;convert 才真正换算钟点。跨市场对齐前必须统一时区')
# BusinessDay:跳过周末的工作日偏移
last_day = daily.index[-1]
next_bday = last_day + pd.offsets.BDay(1)
print(f'最后交易日 {last_day.date()} 后推 1 个工作日 = {next_bday.date()}(自动跳过周末)')

# ===== 6. 出图 =====
# chart_01:某一天的分钟级日内走势
intraday = df.loc[chk_day, 'close']
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(intraday.index, intraday.values, color='#1f77b4', lw=1.3)
ax.set_title(f'{NAME} {chk_day} 日内 5 分钟收盘价走势')
ax.set_ylabel('价格(元)'); ax.grid(alpha=0.3)
fig.autofmt_xdate()
fig.tight_layout(); fig.savefig('chart_intraday.png', dpi=130); plt.close(fig)
```

```
# chart_02:分钟close 全貌 + resample 后的日线close 叠加(点)
fig, ax = plt.subplots(figsize=(11, 5))
```

04 · PYTHON LAB · 真实可跑代码 · 续 (5/5)

时间索引与重采样五件套 — to_datetime / DatetimeIndex / resample 分钟转日线 / OHLCV 聚合 / 时区

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
ax.plot(df.index, df['close'], color='#aaaaaa', lw=0.6, label='5 分钟 close')
ax.plot(daily.index, daily['close'], 'o-', color='#d62728', ms=4, lw=1.2,
label='日线 close(resample)')
ax.set_title(f'{NAME}:分钟数据 resample 成日线(2024-11 ~ 12)')
ax.set_ylabel('价格(元)'); ax.legend(); ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig('chart_resample.png', dpi=130); plt.close(fig)

# chart_03:日内成交量分布(按时段聚合,体现 volume=sum 的意义 + 日内 U 型)
by_time = df.groupby(df.index.strftime('%H:%M'))['volume'].mean()
fig, ax = plt.subplots(figsize=(12, 5))
ax.bar(range(len(by_time)), by_time.values, color='#2ca02c')
step = max(1, len(by_time)//12)
ax.set_xticks(range(0, len(by_time), step))
ax.set_xticklabels(by_time.index[::step], rotation=45, ha='right')
ax.set_title(f'{NAME} 日内各时段平均成交量(经典 U 型:开盘收盘量大)')
ax.set_ylabel('平均成交量'); ax.grid(alpha=0.3, axis='y')
fig.tight_layout(); fig.savefig('chart_intraday_volume.png', dpi=130);
plt.close(fig)

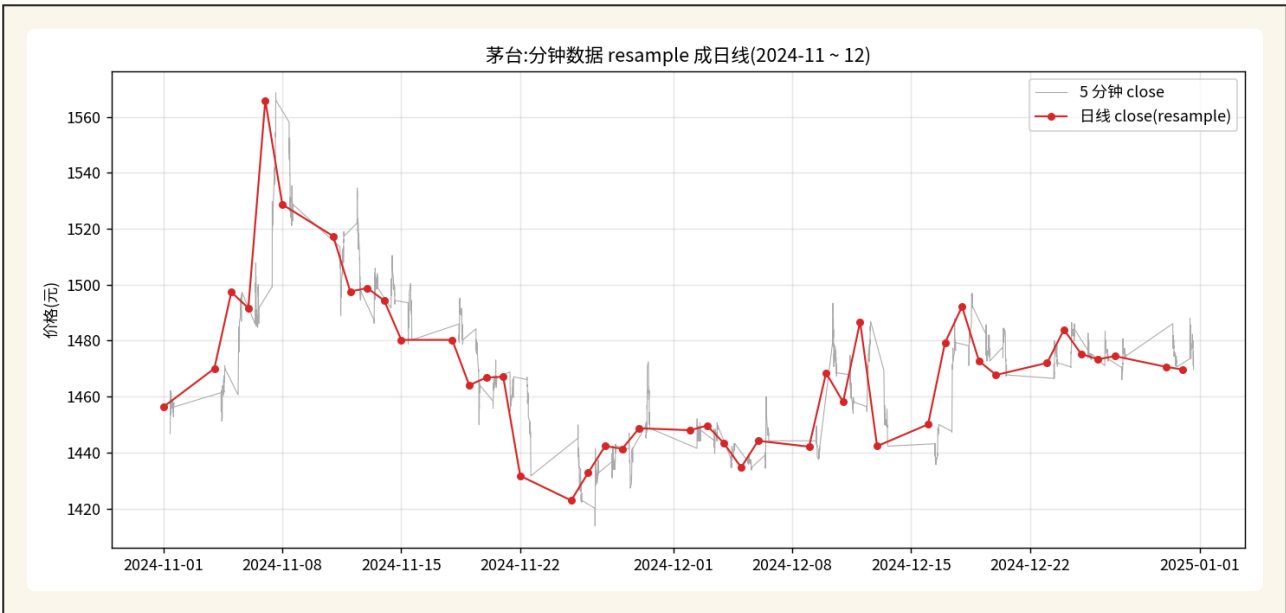
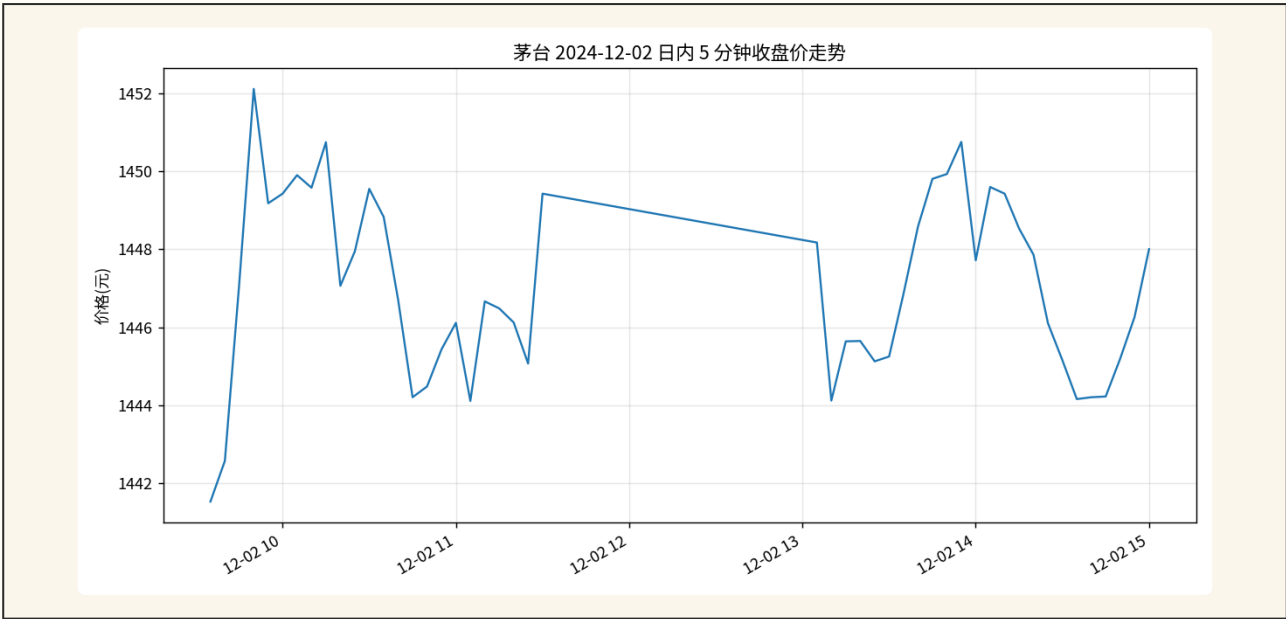
# chart_04:日线 vs 周线收盘价对比
fig, ax = plt.subplots(figsize=(11, 5))
ax.plot(daily.index, daily['close'], '-', color='#1f77b4', lw=1, label='日线')
ax.plot(weekly.index, weekly.values, 's-', color='#ff7f0e', ms=6, lw=1.5,
label='周线')
ax.set_title(f'{NAME}:同一段行情 日线 vs 周线(resample 到不同频率)')
ax.set_ylabel('收盘价(元)'); ax.legend(); ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig('chart_daily_weekly.png', dpi=130);
plt.close(fig)

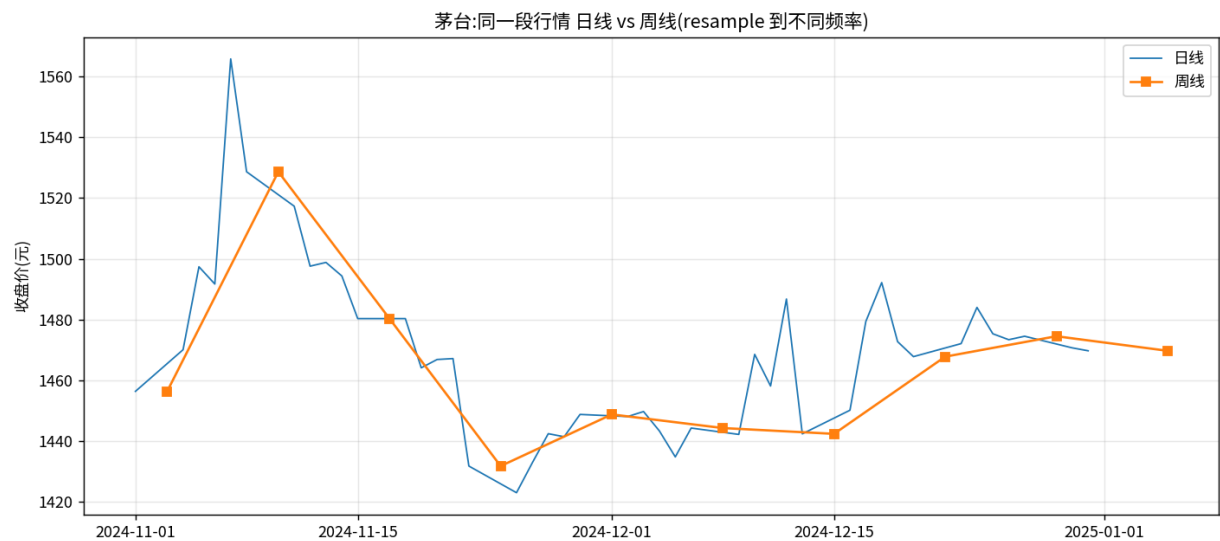
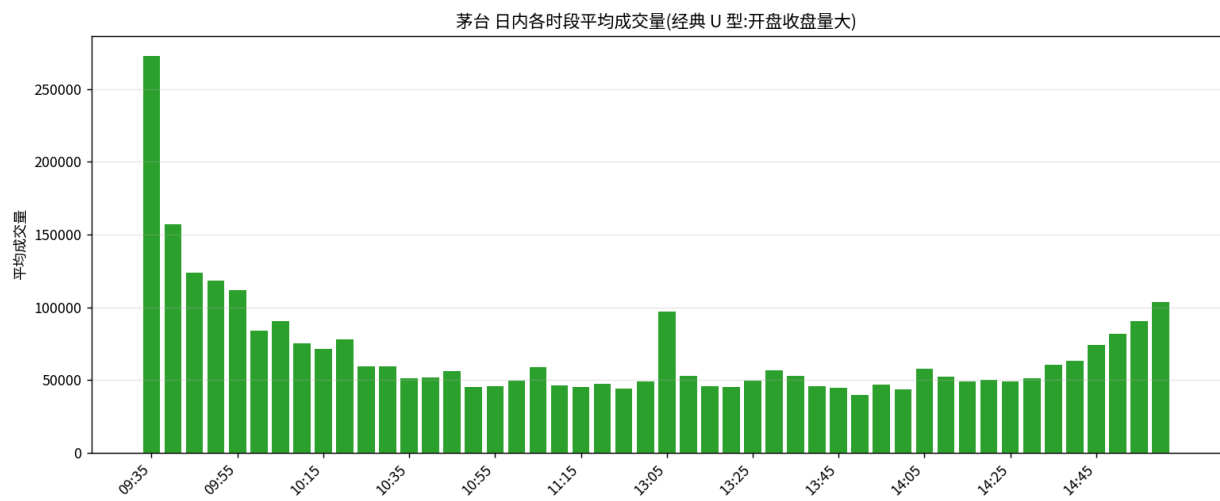
print('\n[done] 4 张图已保存,output.txt 见上方全部打印')
```

05 · CHART + ANALYSIS · 看图说话

回测结果

// · matplotlib 真实输出





原始数据规模

茅台
2024-11 ~
12 的 5 分钟
线共 2064
根,一天 48
根(上午下午各
24 根),12
月有 1056 根

分钟 resample 成日线

2064 根 5
分钟线聚合成
43 根日线;校
验
2024-12-02
当天分钟最高
价 1452.54
= 日线 high
1452.54,完
全一致

聚合方式对错对比
(2024-12-02)

收盘价:正确取
最后一根
1448.00 vs
错误取全天平
均 1447.01,
只差约 1 元
(隐蔽);成交
量:正确求和
268.4 万 vs
错误取平均
5.6 万,差了
约 48 倍(致
命)

日内成交量 U 型

开盘 09:35
巨量约 27
万,一路缩量到
午盘最低约 4
万,13:05 午
后开盘小幅反
弹,尾盘
14:55 回升到
约 10 万

时区换算

贴上北京时区
后,
2024-11-01
09:35 换算成
纽约时间是
2024-10-31
21:35(同一时
刻、差约 13
小时)

工作日偏移

最后交易日
2024-12-31
用工作日偏移
后推一天 =
2025-01-01;
注意它只跳周
末、不跳法定
节假日(元
旦),真实交易
日历要用专门
日历库

ANALYSIS · 这张图告诉我们什么

► 01 取平均的危害非对称:成交量差约 48 倍

聚合方式一刀切取平均的危害是非对称的:收盘价取平均才差约 1 元、看起来好像没事,但成交量取平均跟真实总量差了约 48 倍 — 这正是小陈『放量突破』策略永远不触发的根因,价格的小错能蒙混、成交量的大错直接毁掉所有量价信号。

► 02 成交量日内 U 型:开盘巨量、午盘缩量、尾盘回升

茅台日内成交量是教科书级的 U 型:开盘 09:35 约 27 万一柱擎天,午盘缩到约 4 万,13:05 午后开盘小幅反弹,尾盘回升 — 这条 U 型曲线是市场微观结构的入门一课,也解释了为什么算 VWAP 必须用正确的成交量求和而非平均。

▶ 03 **2064 根精确聚成 43 根日线,最高价校验一致**

2064 根 5 分钟线精确聚合成 43 根日线,且日线最高价严格等于当天所有分钟最高价(1452.54 完全一致)— 证明只要每列用对聚合方式(高 max、低 min、收 last、量 sum),resample 的结果绝对可靠。

▶ 04 **北京 09:35 = 纽约前一天 21:35,差约 13 小时**

同一时刻北京 09:35 等于纽约前一天 21:35,差约 13 小时 — 做 A 股加美股的跨市场对齐时,不先统一时区就会把差半天的数据错误对齐,这是跨市场分析最隐蔽的地基性错误。

▶ 05 **工作日偏移只跳周末、不跳节假日(元旦休市)**

工作日偏移把 2024-12-31 后推一天得到 2025-01-01,但元旦其实休市 — 说明工作日偏移只会跳周末、不认法定节假日,做 T 加一交割这类逻辑必须叠加真实交易日历,不能只靠它。

A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

A 股散户	茅台等任意个股的分钟数据	散户做日内看的是分钟图,但要做策略回测、算「每天涨跌」「每天成交量」,必须先把分钟数据 resample 成日线。这一步聚合方式一旦搞错(比如成交量取了平均),后面所有量价信号全废,而且代码不报错,极其隐蔽。这正是这一节故事的原型,也是每个量化新手都要趟过的坑。
量化研究 / 数据工程	全市场 tick / 分钟行情库	量化机构的数据团队拿到的是最原始的逐笔(tick)或分钟数据,要按需聚合成 1 分钟、5 分钟、15 分钟、日、周等各种周期 K 线供策略调用。resample 加 OHLCV 聚合就是这套数据流水线的核心环节,聚合规则写错一次,下游几十个策略全部中招。
跨市场 / 全球宏观	A 股 + 美股 + 港股联动	研究 A 股跟美股隔夜的联动关系时,A 股是北京时间、美股是纽约时间,差十几个小时。不先用 tz_localize 和 tz_convert 把两边时区统一,就会把不同时刻的数据错误对齐,算出来的相关性、领先滞后关系全是假的。这是跨市场分析最隐蔽的地基性错误。
量价分析 / VWAP	任意活跃股的日内数据	成交量在一天内呈现经典的「U 型」——开盘和收盘前量大、午盘量小。算成交量加权均价 VWAP、判断放量缩量,都依赖对成交量的正确聚合(求和)。按日内时段对成交量分组求平均,就能看出这个 U 型,这是理解市场微观结构的入门一课。
事件 / 财报统计	指数或个股的长周期数据	研究「每月最后一个交易日效应」「季度财报季的波动」等日历效应时,要把日线数据按月('ME')或按季 resample,统计每个周期的收益、波动。这时月末对齐('ME')还是月初对齐('MS')必须选对,否则把财报日归错月份,统计结论就偏了。

这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

△ 01

resample 所有列一刀切取平均 — OHLCV 全错且不报错

对行情数据 resample 时图省事写 .mean() 把所有列都取平均,是最隐蔽的致命错。价格取平均得到的是无意义的全天均价,成交量取平均凭空缩水几十倍,而代码完全不报错。**正确做法:**用 .agg() 配字典,开盘 first、最高 max、最低 min、收盘 last、成交量 sum,每列各管各的,这套 OHLCV 规则要刻进肌肉记忆。

△ 02

忘了 dropna — resample 出一堆周末空行

resample 按自然周期切分,周末、节假日没有交易数据,会产生一堆全是 NaN 的空行。不清理的话,后续画图断断续续、算指标被 NaN 传染。**正确做法:**resample 聚合后顺手 .dropna(how='any') 或 how='all',把没有真实交易的空周期清掉,只留有数据的交易日。

△ 03

tz_localize 和 tz_convert 混用 — 时间错十几个小时

把贴标签的 tz_localize 和换算的 tz_convert 搞混:本该 localize(只贴标签不改钟点)却用了 convert(改了钟点),时间一下错十几个小时。还有 naive(不带时区)和 aware(带时区)的时间直接相减会报错。**正确做法:**裸时间先 localize 贴上本时区,要跨时区比较再 convert;同一组运算里所有时间的时区状态要一致。

△ 04

to_datetime 月日顺序猜错 — 数据悄悄串位

碰到 03/04 这种有歧义的格式,不指定格式让 Pandas 自己猜,它可能把 3 月 4 号猜成 4 月 3 号,而且不报错。一旦猜错,整个时间轴串位,后续全错。**正确做法:**格式有歧义时,显式传 format 参数(如 '%m/%d/%Y')或 dayfirst 参数明确告诉它月日顺序,绝不靠猜。

△ 05

resample 周期标签用错 — 月末月初差半个月

按月 resample 时,'ME' 是对齐到月末最后一天、'MS' 是对齐到月初第一天,'W' 默认对齐到周日。标签选错,聚合出来的日期会整体偏移半个月或几天,把数据归错周期。**正确做法:**想清楚要月末还是月初对齐,查准频率别名;聚合后打印几行索引,确认日期落在预期位置。

06 · STANDARD OPERATING PROCEDURE · 实战规则

时间索引与重采样实战 7 条 SOP

// 按这套规则走,你的执行力会立刻拉到中等机构水平

- 01** 行情数据进门第一步:时间列 `pd.to_datetime` 解析 + `set_index` 成 `DatetimeIndex` — 一切时间操作的前提
- 02** 格式怪或有歧义的时间显式给 `format` 参数 — 又准又快,杜绝月日顺序猜错
- 03** `resample` 降频是量化最高频操作 — 'D' 按天、'W' 按周、'ME' 按月末,想清楚对齐到哪
- 04** OHLCV 聚合用 `.agg` 配字典 — 开 `first` / 高 `max` / 低 `min` / 收 `last` / 量 `sum`,绝不一刀切 `mean`
- 05** `resample` 后顺手 `dropna` — 清掉周末节假日产生的空周期
- 06** 跨市场对齐前先统一时区 — 各自 `tz_localize` 到本时区,再 `tz_convert` 到同一时区
- 07** 往后推交易日用 `pd.offsets.BDay` 不用加一天 — 自动跳过周末,T+1 交割/次日执行都靠它

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

- 01 DatetimeIndex 是时间专用索引 — 支持部分字符串索引(给月份选整月、给一天选一天),还能直取小时/星期做日内分析
- 02 pd.to_datetime 把各种时间字符串解析成标准时间戳 — 格式怪或有歧义显式给 format,杜绝月日顺序猜错
- 03 resample 是时间版 groupby — 按周期把数据切段聚合,最常用于把分钟降频成日线/周线/月线
- 04 OHLCV 聚合规则是铁律 — 开 first / 高 max / 低 min / 收 last / 量 sum,价格不能取平均、成交量不能取最后一个
- 05 一刀切取平均是最隐蔽的坑 — 代码不报错但量价信号全废,小陈的策略永远不触发就栽在这
- 06 时区:tz_localize 贴标签不改钟点、tz_convert 换算钟点看同一时刻 — 跨市场对齐前必须统一时区
- 07 工作日偏移用 pd.offsets.BDay — 往后推交易日自动跳过周末,T+1 交割、信号次日执行靠它

08 · SELF-CHECK · 自测题

答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

Q01 DatetimeIndex 比普通文字索引多了哪些超能力?写 `df.loc['2024-12']` 会选出什么?(提示:部分字符串索引、时间属性直取、支持 `resample/` 时区;选出整个 12 月的所有行)

Q02 `resample` 是什么?它跟 `groupby` 有什么类比关系?把分钟收盘价聚合成日收盘价该怎么写?(提示:时间版 `groupby`,按时间段分组聚合;`df['close'].resample('D').last()`)

Q03 OHLCV 五个字段各自该用什么聚合方式?为什么成交量不能取平均?(提示:开 `first/` 高 `max/` 低 `min/` 收 `last/` 量 `sum`;日成交量是当天所有 K 线之和,取平均会缩水几十倍)

Q04 `tz_localize` 和 `tz_convert` 有什么区别?做 A 股加美股跨市场对齐时为什么必须先处理时区?(提示:`localize` 贴标签不改钟点、`convert` 换算钟点;不统一时区会把差十几小时的数据错误对齐)

Q05 为什么 `resample` 之后通常要 `dropna`?往后推一个交易日为什么不能简单加一天?(提示:周末节假日产生空周期;加一天可能撞周末,要用 `BDay` 自动跳过)

09 · NEXT STEP · 下一步

继续往前走

// 看完这节,下面是延伸方向 + 明天预告

推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ Wes McKinney 《Python for Data Analysis》(2022 第 3 版,O'Reilly)– 第 11 章专讲时间序列、DatetimeIndex、resample 与时区,Pandas 作者亲笔,本节最对口的权威章节
- ▶ Pandas 官方文档《Time series / date functionality》(免费在线,pandas.pydata.org)– resample 频率别名、offset、时区处理的权威参考,频率字符串('D'/'W'/'ME')查这里最准
- ▶ Jake VanderPlas 《Python Data Science Handbook》(2016,jakevdp.github.io 免费在线)– 第 3 章 Working with Time Series 一节,用真实数据讲 resample,通俗易懂
- ▶ Marcos López de Prado 《Advances in Financial Machine Learning》(2018,Wiley)– 第 2 章讲金融数据的「信息驱动 Bar」(成交量 Bar、美元 Bar),是 resample 思想在量化里的高阶延伸
- ▶ Python 工具栈 – Pandas(resample 核心)、pytz / zoneinfo(时区数据库)、pandas-market-calendars(各国交易所交易日历,处理节假日)、mplfinance(画 K 线图),这四个是时间序列处理的进阶生态

NEXT LECTURE · 下一节预告

DAY 036 滚动窗口 rolling

第 36 课 滚动窗口 rolling。今天学会了把数据聚合成不同周期,明天讲怎么在时间序列上「开一个移动的窗口」做计算 —— 这就是 rolling。你天天听到的 20 日均线、布林带,本质都是 rolling 滚动窗口算出来的。我们会讲 rolling 滚动、expanding 累计、ewm 指数加权三兄弟,亲手实现均线和布林带,还有怎么用 min_periods 处理开头的 NaN。这是所有技术指标的共同地基,学完你就能自己造任何指标。